JOHANNES KEPLER
UNIVERSITY LINZ

# Modeling Guidelines for Synthesizable RTL



Simon Dorrer

**KV VLSI Design, 336.051, WiSe 26/27**

IICQC
INSTITUTE FOR
INTEGRATED CIRCUITS &
QUANTUM COMPUTING

**JOHANNES KEPLER**
**UNIVERSITÄT LINZ**
Altenberger Straße 69
4040 Linz, Österreich
jku.at

# Today's Goals

After this lecture, you can

- explain why the **RTL and the netlist** must simulate alike
- predict how simulators treat the **unknown value x**
- find a **missing reset** before the chip does
- replace **FPGA habits** by ASIC practice
- replace a **divided clock** by a **clock enable**
- design **macro boundaries** that are easy to close and verify
- compare **architectures** by area and timing



Chapter 3

Lecture notes: Chapter 3 of iic-jku.github.io/vlsi-design

# Simulation Must Match the Netlist

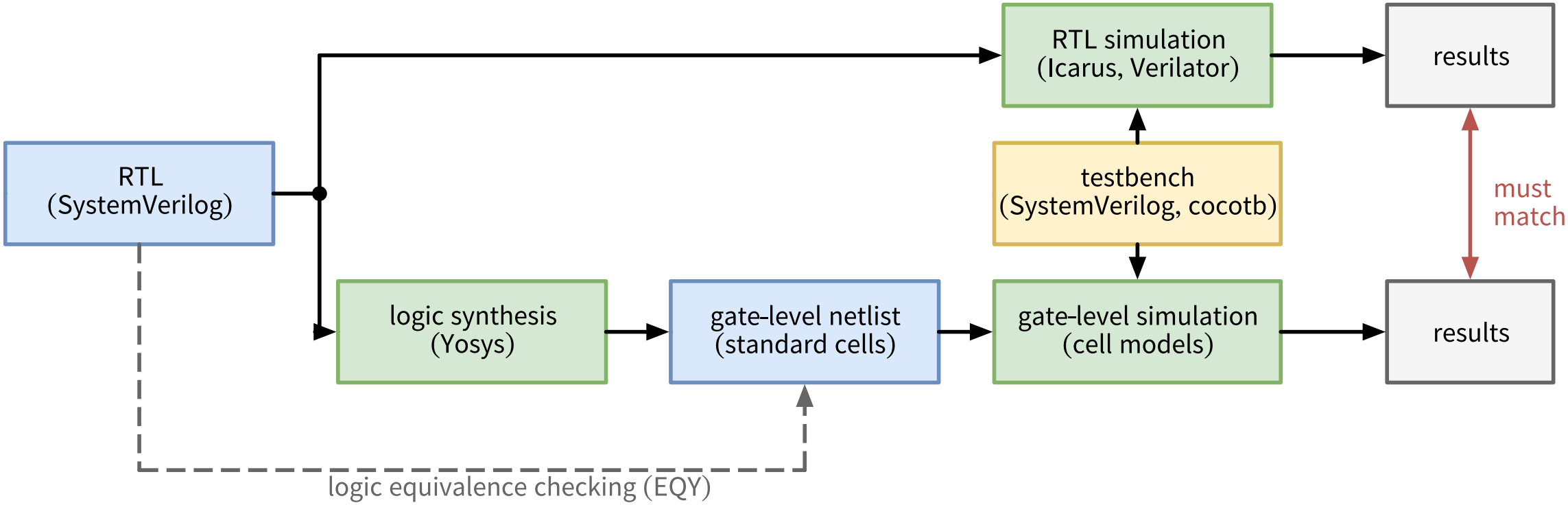


# Levels of Abstraction

| Level | Describes | Example |
|---|---|---|
| **behavioral** | the function only | Python model, testbench |
| **RTL** | registers and the logic between them | your SystemVerilog |
| **gate level** | a netlist of standard cells | output of Yosys |
| **layout** | polygons on the masks | GDSII |

- You write and simulate the **RTL**
- The flow turns it into the **netlist**, the netlist into the **layout**

# The Chip Implements the Netlist




- Same testbench on **RTL and netlist**, the results **must match**
- Logic equivalence checking **proves** it without simulation (Chapter 5)

# Where Mismatches Come From

- Debugging a netlist is hard: generated names, no FSM states, flat hierarchy
  - template counter, final netlist: the flip-flop outputs are `net3` to `net10`
- **Simulation semantics** that synthesis ignores: `x`, initial values
- **Constructs tools read differently**: sensitivity lists, `full_case`, `unique` without `default` (Chapter 2)
- **Timing** the RTL simulation does not model: glitches, derived clocks
- So: write RTL that **cannot** cause a mismatch

# Four-Valued Simulation, Two-Valued Silicon



# 0, 1, x, z

- `logic` has four values: `0`, `1`, `z` (high impedance), `x` (unknown)
- Silicon has two: every node is **0 or 1**, `x` means the **simulator does not know**
  - register before its first load, two drivers, `'x` assignment, index out of range
- **X-optimism**: a known value where the hardware could have either
  - `if` takes the `else` branch, `case` runs the `default`
- **X-pessimism**: `x` where the hardware has a known value
  - e.g., a `default` that assigns `'x` (next slide)

# Experiment: A Multiplexer with sel = x

```
always_comb
  if (sel) y_if = b;
  else     y_if = a;
always_comb
  case (sel)
    1'b0:    y_case = a;
    1'b1:    y_case = b;
    default: y_case = 1'bx;
  endcase
assign y_cond = sel ? b : a;
assign y_and  = sel & a;
```

| sel = x | Icarus | Verilator | Hardware |
|---|---|---|---|
| **if, else** | 0 / 1 | 0 / 1 | 0 or 1 / 1 |
| **case, default x** | x / x | 0 / 1 | 0 or 1 / 1 |
| **sel ? b : a** | x / 1 | 0 / 1 | 0 or 1 / 1 |
| **cell mux2_1** | x / 1 | not run | 0 or 1 / 1 |
| **sel & a** | 0 / x | 0 / 0 | 0 / 0 or 1 |

each entry: a = 0, b = 1 / a = b = 1

Testbench: `xprop/testbenches/verilog/xprop_tb.sv`

# What the Table Shows

- **`if` is optimistic**: shows `0` where the hardware has `0` or `1`
- **An `'x` default is pessimistic**: `x` although the hardware gives `1`
- **`?:` behaves like the mux2_1 cell**: equal bits stay known, different bits become `x`
- **The AND gate is exact** in Icarus: `x & 0 = 0`
- **Verilator knows no x**: two-state, here every `x` became `0`
  - `+verilator+rand+reset+2 +verilator+seed+<n>`: random start values

# Guidelines for x

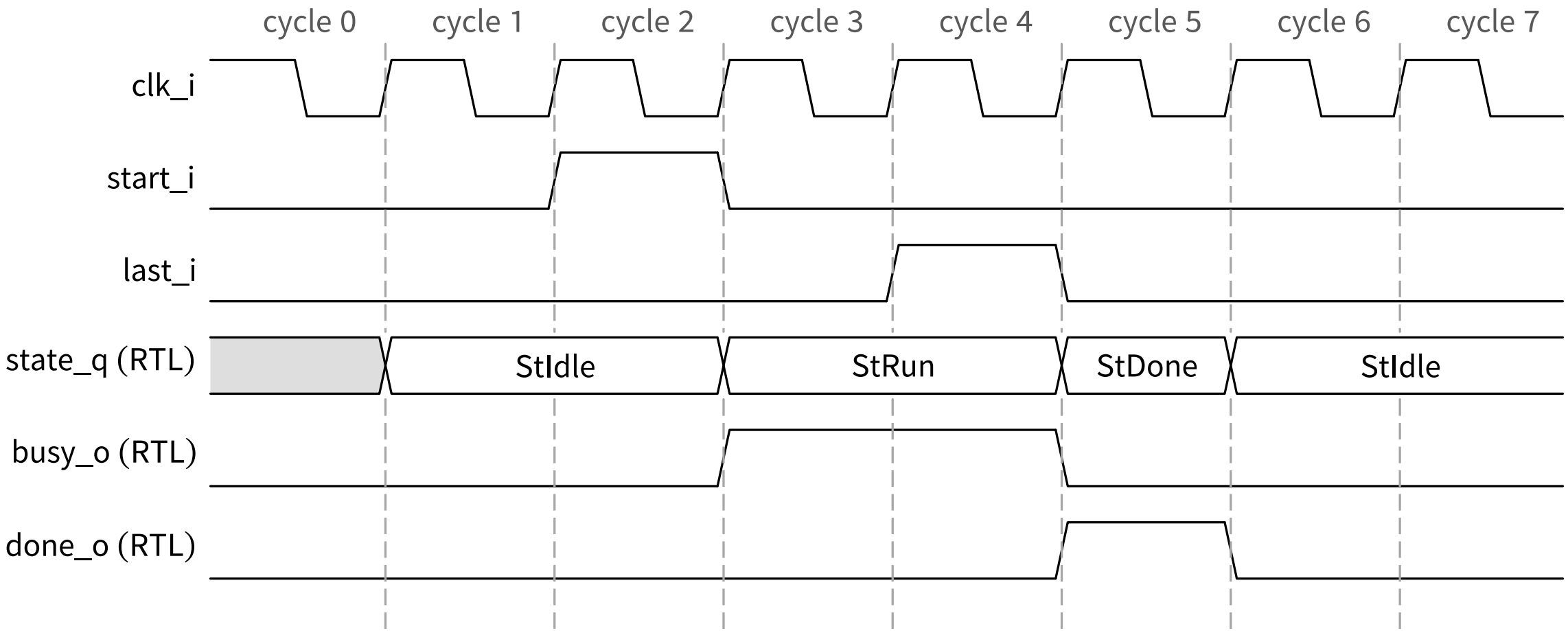
- **Never assign `'x` in RTL**
  - lowRISC: “The use of X literals in RTL code is strongly discouraged”
  - synthesis may pick any value, the netlist differs from the simulation
- **No x in the RTL simulation is no proof**
- Run Verilator with **random start values** and several seeds
- Simulate the **netlist** at least once
- Check with `!==` or `$isunknown()` in testbenches

# The Missing Reset

```
// State register without a reset
always_ff @(posedge clk_i) begin
  state_q <= state_d;
end
```

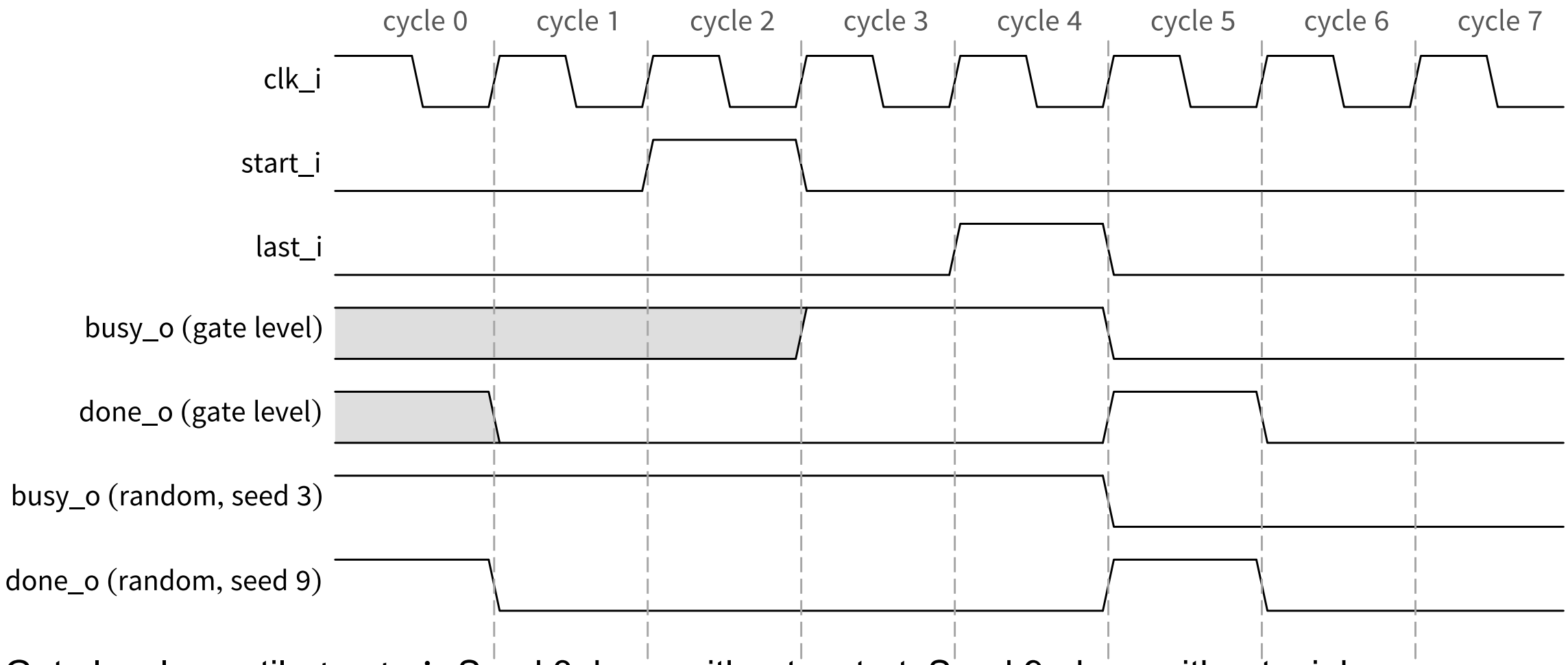
- The controller FSM of Chapter 2, **reset removed**
- `start_i` in cycle 2, `last_i` in cycle 4
- Four simulations of the **same RTL**:
  - Icarus Verilog
  - Verilator
  - Verilator with **random start values**
  - Icarus Verilog, **netlist** on SG13CMOS5L cells

# The RTL Simulation Hides It




No item matches `xx`, the `default` branch returns to `StIdle`: the outputs **never show x**

# The Netlist and Random Start Values Show It




Gate level: `x` until `start_i`. Seed 3: busy without a start. Seed 9: done without a job.

# Live Demo: Four Simulations

```
cd content/modeling/code/ctrl_fsm_noreset
iverilog -g2012 -o rtl.vvp rtl/*.sv testbenches/verilog/*.sv
vvp rtl.vvp                                    # Icarus Verilog
verilator --binary --timing --timescale 1ns/1ps \
  --top-module ctrl_fsm_noreset_tb rtl/*.sv testbenches/verilog/*.sv
./obj_dir/Vctrl_fsm_noreset_tb                 # Verilator
./obj_dir/Vctrl_fsm_noreset_tb +verilator+rand+reset+2 +verilator+seed+3
yosys -p "read_verilog -sv rtl/ctrl_fsm_noreset.sv; synth ...; write_verilog nl.v"
iverilog -g2012 -gno-specify -DGL ... nl.v testbenches/verilog/*.sv
vvp a.out                                      # gate level
```

- Complete commands: exercise in Section 3.3.1

# Guidelines for Resets

- **Reset every register with control state**: FSM states, counters, valid flags, pointers
- A **data register** may stay without reset if it is written before it is read
  - saves the reset net, not cell area: every SG13CMOS5L flip-flop has a reset input
- Use the four-state **`logic`**, also as the base of an enumeration
  - `bit` and `int` start at `0` and hide a missing reset, like Verilator
- VHDL: an enumeration starts with its **leftmost value**, the same trap

**Exercise** (Section 3.3.1): find the missing reset in four simulations.

# FPGA Habits That Break on an ASIC



# FPGA Habits and Their ASIC Counterparts

| FPGA habit | On an ASIC | Instead |
|---|---|---|
| **initial values** | dropped by synthesis | a reset |
| **register output as clock** | a second clock and tree | a clock enable |
| **inferred block RAM** | flip-flops, 10× the area | an SRAM macro |
| **inferred multiplier (DSP)** | a large, slow cell array | a suitable architecture |
| **internal tri-state bus** | can short or float | a multiplexer |
| **PLL, clock manager** | not in the cell library | analog IP, clock gating |
| **global reset at configuration** | no automatic reset | reset pin, synchronizer |

# Initial Values

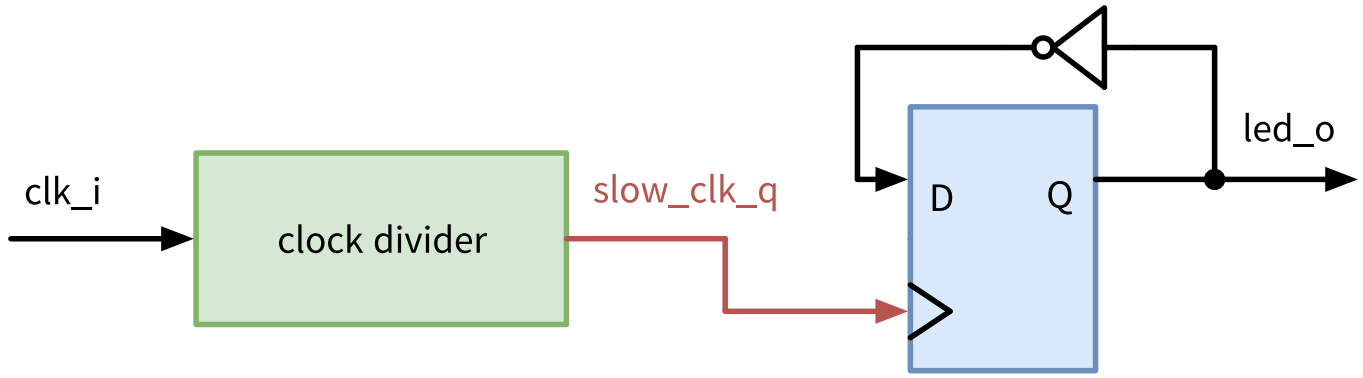
```
logic [COUNT_WIDTH-1:0] count_q    = '0;
logic                   slow_clk_q = 1'b0;
logic                   led_q      = 1'b0;
```

- **RTL simulation** honors them: Icarus and Verilator start at zero
- **Verilator `-Wall`** warns about them, see the exercise
- **Yosys** drops them when mapping onto SG13CMOS5L cells, **without a warning**
- **Netlist** (`DIV = 4`): `led_o` is still x after 40 cycles, the chip starts **random**
- `initial` is for testbenches only, the **reset** sets start values

# Why a Divided Clock Hurts

- A second **clock**: every path between the two must be timed
- A second **clock tree**, skewed against `clk_i`
- A **generated clock** in the constraints (Chapter 6)
- The **scan test** must control it (Chapter 9)
- Without reset: x, **no edge** at all

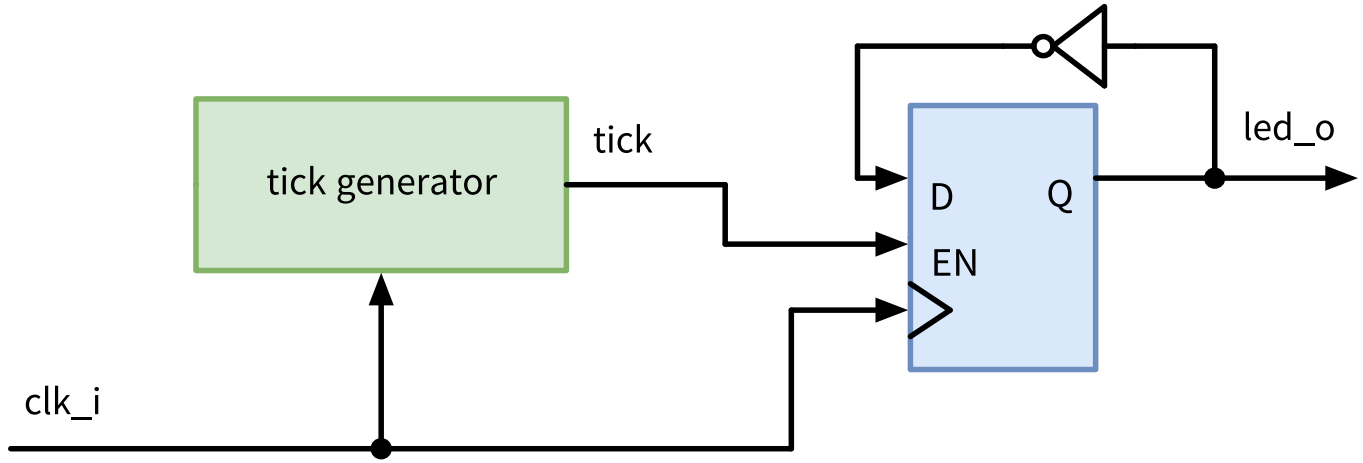
(a) divided clock: two clock domains



# The Clock Enable

- Every **rising edge** of the slow clock becomes a **one-cycle `tick`**
- All registers stay on **`clk_i`**
- One clock, one tree, one set of constraints
- Idle blocks: switch the clock off with **clock gating cells** (Chapter 7)

(b) clock enable: one clock




**Exercise** (Section 3.4.2): an LED blinker from FPGA to ASIC.

# Memories: Flip-Flops or an SRAM Macro?

| 256 words × 8 bits | Cell area | Per bit |
|---|---|---|
| **flip-flops and multiplexers (Yosys)** | 165,743 µm² | 81 µm² |
| **SRAM macro RM_IHPSG13_1P_256x8_c3_bm_bist** | 17,547 µm² | 8.6 µm² |

- Flip-flops: 2056 `dfrbpq_1`, 2074 `mux2_1`, 368 `mux4_1`, **9.4× the area**
- Even 32 × 8 bits (20,944 µm²) exceed the smallest SRAM, 64 × 16 (15,240 µm²)
- **Flip-flops**: small FIFOs, register files with several read ports
- **SRAM macro**: everything larger, a hard macro with model, timing, LEF, GDS

# Tri-States

```
tri [7:0] bus;
assign bus = sel_a_i  ? a_i : 8'bz;  // driver A
assign bus = !sel_a_i ? b_i : 8'bz;  // driver B
assign y_o = bus;
```

- Verilator: no warning. Plain Yosys `synth`: **error**, `y_o` “already driven”
- LibreLane maps it onto `sg13cmos5l_ebufn_2` tri-state buffers
- Still avoid it: overlapping enables **short**, no enable **floats**
- lowRISC: “Do not use three-state logic … to accomplish on-chip logic”
- Write a **multiplexer**, tri-states only in the **pads**

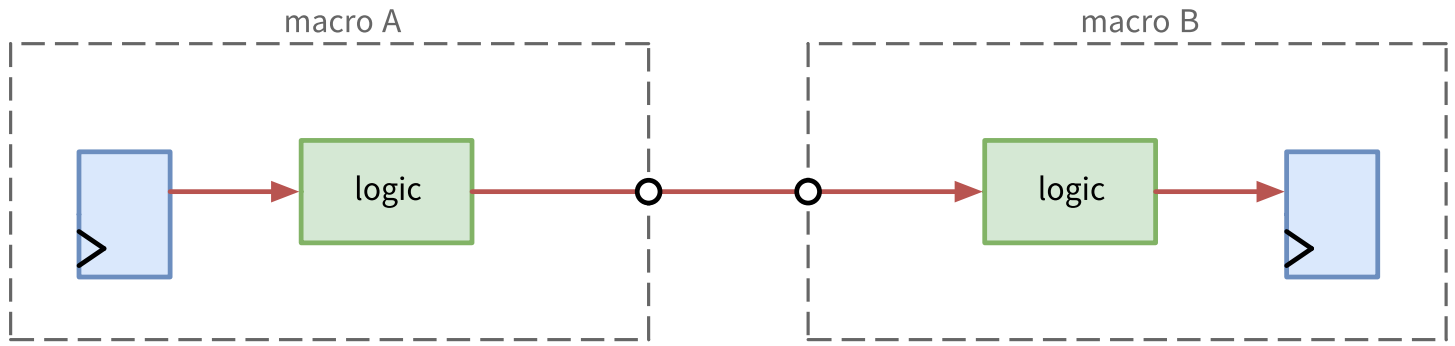
# Macro Boundaries



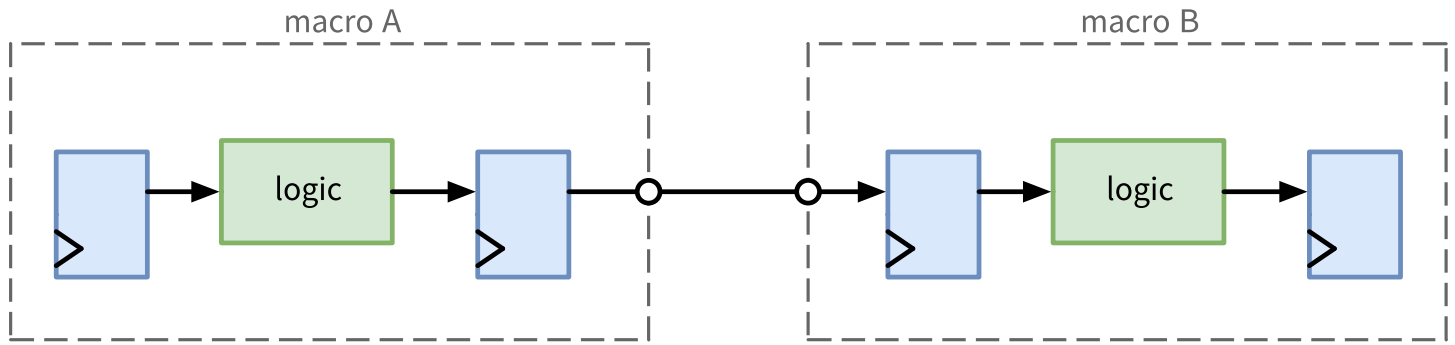
# Registered Inputs and Outputs

- Macros are **hardened separately**
- LibreLane assumes **20 %** of the period outside
  - 4 ns of the 20 ns of the counter
- Register every **output**, inputs from far away too
- Price: a cycle of **latency**

(a) unregistered boundary




(b) registered outputs and inputs



# Ports That Survive Synthesis

- Same cocotb testbench at RTL and gate level: `make sim-gl-cocotb`
- **Plain `logic` vectors** at the top-level ports
- **No top-level parameters** that the testbench changes
- **Access only the ports**: internal names do not survive
- **A reset** at the start of every test, the netlist starts as `x`

# Architecture Determines the Result



# Synthesis Keeps Your Architecture

- Synthesis **optimizes the logic** you describe
- It does **not invent** a new architecture
  - a binary counter stays a binary counter, a shift register a shift register
- The **architecture** decides area and speed, much more than the code
- Case study: the **exclusive counter**

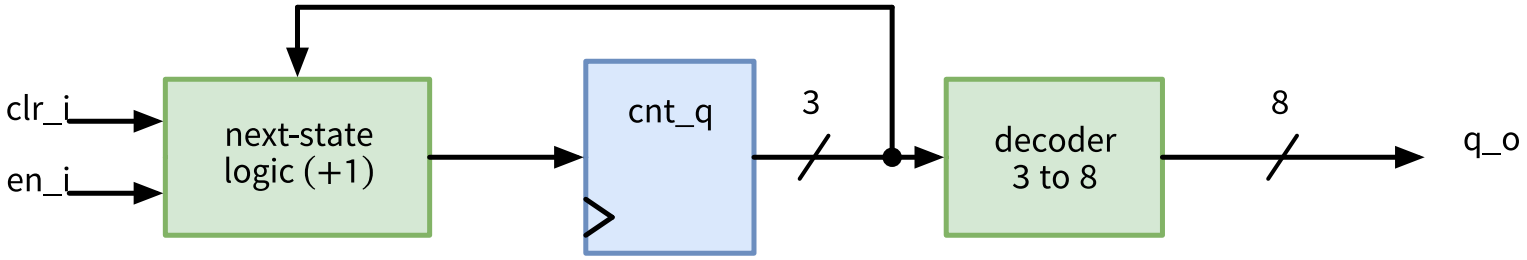
# The Exclusive Counter

- Eight outputs `q_o[7:0]`, **exactly one is high** (one-hot)
- Reset `rst_ni` and synchronous `clr_i` (priority over `en_i`): `q_o[0]` high
- `en_i`: the high bit moves up by one, **wraps** from `q_o[7]` to `q_o[0]`
- Otherwise `q_o` keeps its value

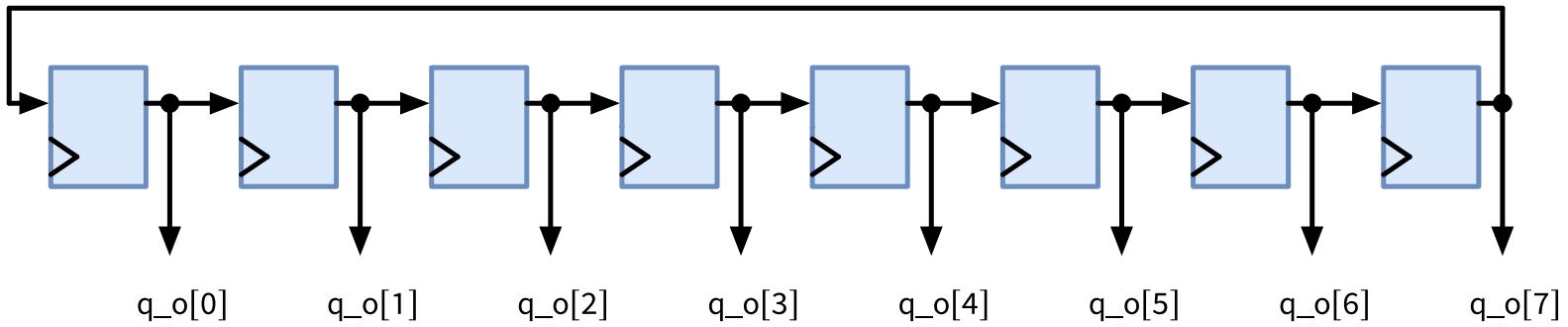
```
q_o: 00000001 -> 00000010 -> 00000100 -> ... -> 10000000 -> 00000001
```

# Two Architectures

(a) binary counter and decoder




(b) one-hot ring



# Two Descriptions

Binary counter and decoder

```
always_comb begin
  cnt_d = cnt_q;
  if (clr_i) begin
    cnt_d = '0;
  end else if (en_i) begin
    cnt_d = cnt_q + 3'd1;
  end
end
assign q_o = 8'b0000_0001 << cnt_q;
```

One-hot ring

```
always_comb begin
  q_d = q_q;
  if (clr_i) begin
    q_d = 8'b0000_0001;
  end else if (en_i) begin
    q_d = {q_q[6:0], q_q[7]};
  end
end
assign q_o = q_q;
```

- Same register process with `rst_ni`, same cocotb testbench: both pass

# Results on SG13CMOS5L

| | Binary + decoder | One-hot ring |
|---|---|---|
| **flip-flops** | 3 | 8 |
| **standard cells** | 22 | 28 |
| **cell area** | 308.4 µm² | 553.2 µm² |
| **clock to output** | 0.49 ns | 0.23 ns |
| **flip-flop to flip-flop** | 0.52 ns | 0.31 ns |

typical corner, 10 fF load, ideal clock (Yosys, OpenSTA)

- Ring: **+79 % area**
- Ring: **faster**, clean outputs
- Binary: one-hot **by construction**
- `for` loop variant of the ring: **identical** netlist

# A Third Architecture

- **Johnson counter**: 4 flip-flops, shift left, feed back the inverted MSB `~s_q[3]`
- Eight states, **one bit changes** per counting step
- Decoder: one **2-input gate** per output
- Where does it rank in area and speed?

**Exercise** (Section 3.6.1): a third architecture, the Johnson counter.

# Checklist and Summary



# Checklist for ASIC-Ready RTL

- Reset all **control state**
- No **initial values**, no **'x**
- **One clock**: a clock enable, no derived clocks
- Larger memories as **SRAM macros**
- No internal **tri-states**
- **Registered** macro outputs, **plain** top-level ports
- The **architecture** first, measured with Yosys and OpenSTA
- Simulate in **Icarus**, **Verilator** with random start values, and at **gate level**

# Summary

- You verify the RTL, the chip implements the **netlist**: they must agree
- `if` and `case` misjudge `x`, `?:` matches the mux2_1 cell, **Verilator shows no x**
- A **missing reset** hides in RTL: random start values and the netlist reveal it
- **FPGA habits** do not carry over: initial values, divided clocks, inferred RAMs, tri-states
- **Clock enable** instead of a divided clock, **registered** macro outputs
- The **architecture** decides area and speed, not the coding style

# Exercises

1. Find the Missing Reset (3.3.1)
2. An LED Blinker from FPGA to ASIC (3.4.2)
3. A Third Architecture, the Johnson Counter (3.6.1)

All code in `content/modeling/code` of the course repository, every number reproduced by `scripts/check_modeling.sh`



Chapter 3



JOHANNES KEPLER
UNIVERSITY LINZ